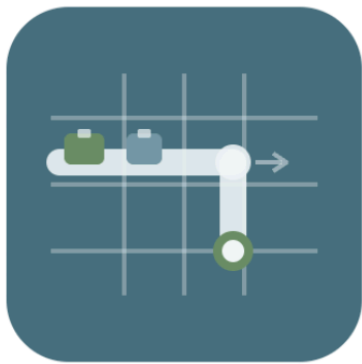


Architekturdokument

V 1.0



SortiFX
MODELLIERUNG & SIMULATION

Projektbezeichnung	SortiFX - Modellierungs- und Simulationssoftware für Gepäcksortieranlagen
Projektleiter	Shiraz Ahmad, Angelina Alpermann-Mahr, Lilian Do

Bezug auf Anforderungsspezifikation V 1.0

Versionshistorie

Version & Datum	Änderungen	Ansprechpersonen
V1.0 - 16.06.26	erste veröffentlichte Version des Dokuments	Projektleiter

Inhaltsverzeichnis

Inhaltsverzeichnis.....	3
1. Einführung und Ziele.....	4
1.1 Aufgabenstellung.....	4
1.2 Qualitätsziele.....	4
1.3 Stakeholder.....	4
2. Randbedingungen.....	5
3. Kontextabgrenzung.....	5
3.1 Fachlicher Kontext.....	5
3.2 Technischer Kontext.....	5
4. Lösungsstrategie.....	6
5. Bausteinsicht.....	7
5. 1 Paketdiagramm.....	7
Beschreibung Paketdiagramm.....	8
5. 2 Verfeinerungsebenen.....	9
Beschreibung Verfeinerungsebenen.....	10
6. Laufzeitsicht.....	11
6.1 Szenario: Förderband-Element platzieren.....	11
6.2 Szenario: Förderweg validieren.....	13
6.3 Szenario: Weiche konfigurieren.....	14
7. Verteilungssicht.....	15
8. Querschnittliche Konzepte.....	15
8.1 MVC.....	15
8.2 Validierungskonzept.....	15
8.3 Persistenzkonzept.....	15
9. Entwurfsentscheidungen.....	15
9.1 MVC.....	15
9.2 JavaFX.....	16
9.3 Validierung als eigene Klasse.....	16
10. Qualitätsszenarien.....	16
11. Risiken & technische Schulden.....	16
12. Glossar.....	17

1. Einführung und Ziele

1.1 Aufgabenstellung

Die fachlichen Ziele und Anforderungen des Systems werden in der *Anforderungsspezifikation Kapitel 1.1 beschrieben*. Die Architekturdokumentation konzentriert sich auf die technische Umsetzung dieser Anforderungen.

1.2 Qualitätsziele

Die fachlichen Qualitätsanforderungen des Systems sind in *Kapitel 4.1 der Anforderungsspezifikation* beschrieben.

Für die Architektur von SortiFX stehen insbesondere folgende Qualitätsziele im Vordergrund:

Qualitätsziele	Beschreibung
Wartbarkeit	Die Software soll klar in Schichten und fachliche Komponenten getrennt sein.
Testbarkeit	Simulationslogik, Routing und Validierung sollen unabhängig von JavaFX testbar sein.
Erweiterbarkeit	Neue Förderband-Elemente, Gepäckarten, oder Routing Regeln sollen später ergänzbar sein.
Nutzbarkeit	Die Bedienung soll über eine verständliche grafische Oberfläche erfolgen
Zuverlässigkeit	Fehlerhafte Förderwege und ungültige Zustände sollen erkannt und nachvollziehbar angezeigt werden.

1.3 Stakeholder

Stakeholder	Interesse
Nutzer/in	Förderanlagen erstellen, konfigurieren, simulieren
Entwicklungsteam	Verständliche, wartbare und testbare Architektur
Prüfer/Dozent	Nachvollziehbare technische Umsetzung der Anforderung
Flughafenlogistik / Planung	Simulation und Analyse von Gepäckförderanlagen

2. Randbedingungen

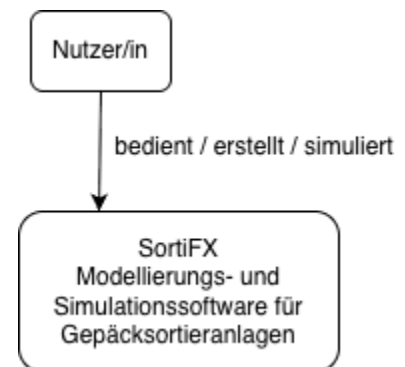
Die technischen und organisatorischen Randbedingungen, insbesondere Java 21, JavaFX, Gradle und Linux als Zielplattform, werden in der *Anforderungsspezifikation Kapitel 1.2* definiert und werden von dieser Architektur übernommen.

3. Kontextabgrenzung

3.1 Fachlicher Kontext

Die fachlichen Anforderungen und Anwendungsfälle werden in *Kapitel 2 der Anforderungsspezifikation* beschrieben. Das dort definierte *Domänenmodell* in *Kapitel 3.1* bildet die fachliche Grundlage der vorliegenden Architektur.

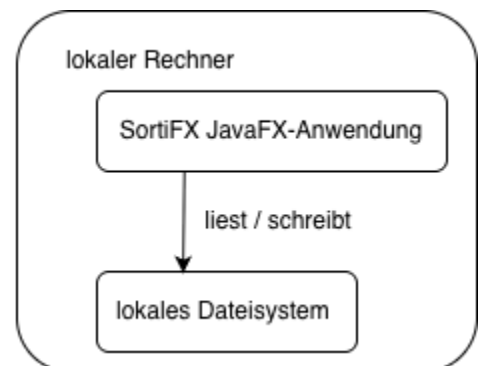
SortiFX wird fachlich von einer Nutzerin bzw. einem Nutzer bedient. Diese erstellen und bearbeiten Anlagenpläne, platzieren Förderband-Elemente und Gepäckstücke, konfigurieren Weichen, validieren Förderwege und starten Simulationen. Das System stellt dafür eine grafische Modellierungs- und Simulationsumgebung bereit.



3.2 Technischer Kontext

SortiFX wird als lokale Desktop-Anwendung entwickelt. Die Benutzeroberfläche basiert auf JavaFX. Die Anwendungslogik, Simulationslogik und Persistenz werden innerhalb derselben Anwendung ausgeführt.

Zur Speicherung und zum Laden von Anlagenplänen greift die Anwendung auf das lokale Dateisystem zu. Eine Netzwerkkommunikation oder externe Dienste werden nicht benötigt.



4. Lösungsstrategie

Die Architektur von SortiFX verfolgt das Ziel, Benutzeroberfläche, Steuerungslogik, fachliches Modell und Persistenz klar voneinander zu trennen. Dadurch sollen die Wartbarkeit, Testbarkeit und Erweiterbarkeit der Anwendung verbessert werden.

Die Anwendung wird als JavaFX-basierte Desktop-Anwendung umgesetzt. Die grafische Benutzeroberfläche befindet sich in der **view**-Schicht und dient ausschließlich der Darstellung sowie der Weiterleitung von Benutzereingaben. Fachliche Abläufe werden nicht direkt an der Oberfläche umgesetzt.

Die **controller**-Schicht vermittelt zwischen Benutzeroberfläche und fachlichem Modell. Sie nimmt Aktionen aus der view-Schicht entgegen, ruft die passenden Funktionen im **model** auf und gibt Ergebnisse an die Oberfläche zurück.

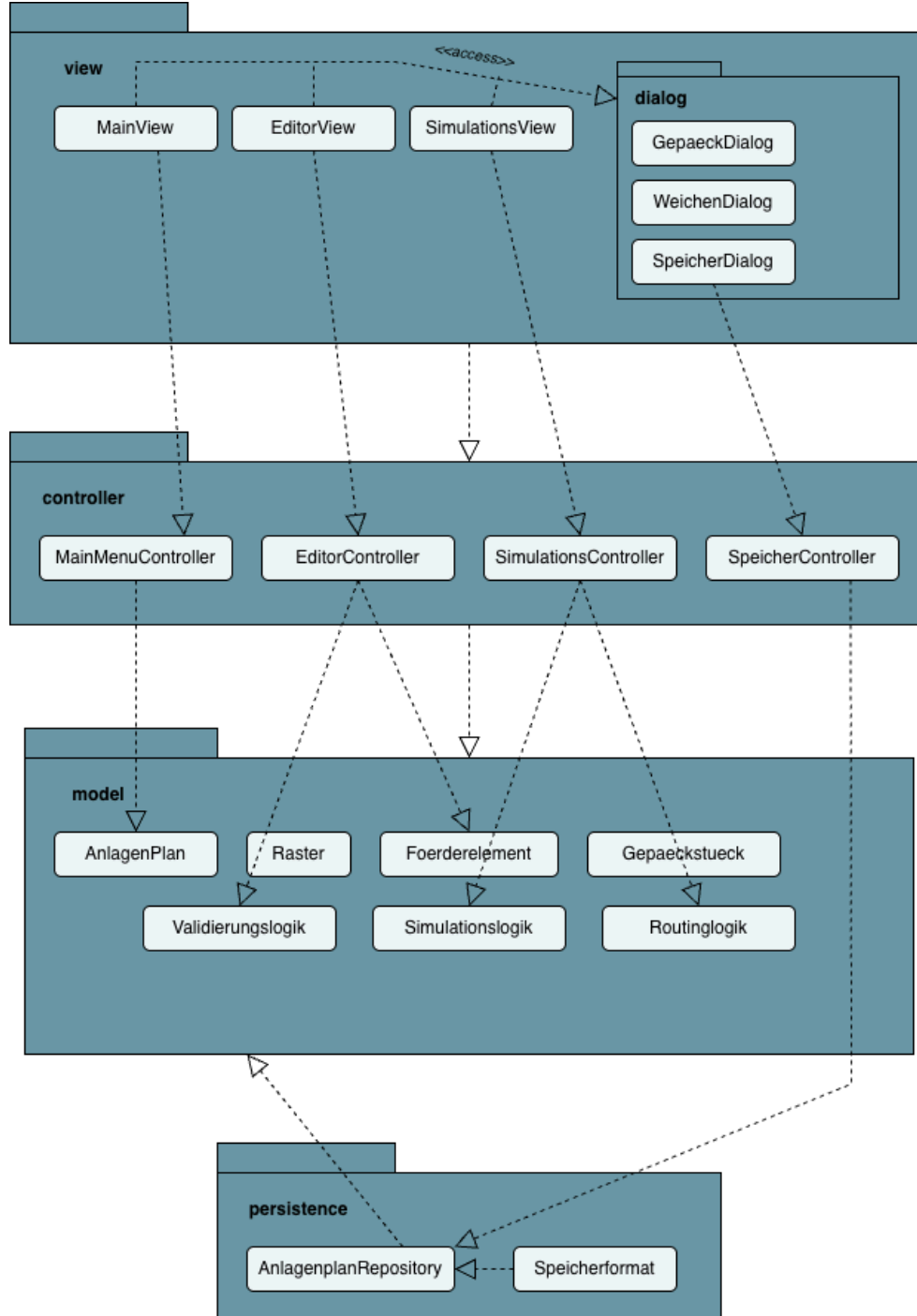
Das **model** enthält die zentralen Domänenklassen der Anwendung, insbesondere Anlagenplan, Raster, Förderband-Elemente und Gepäckstücke. Zusätzlich befinden sich dort die UI-neutralen Kernlogiken für Validierung, Simulation und Routing. Diese Logiken sind unabhängig von JavaFX und können daher separat getestet werden.

Die **persistence**-Schicht kapselt das Speichern und Laden von Anlagenplänen. Sie stellt eine klare Schnittstelle zum Dateisystem bereit und verhindert, dass Speicherlogik direkt in View-, Controller- oder Model-Klassen verteilt wird.

Damit folgt die Architektur einer schichtenartigen Struktur mit den Paketen **view**, **controller**, **model** und **persistence**. Die Abhängigkeiten verlaufen von oben nach unten: Die view kennt controller, controller kennt model und persistence, während das model unabhängig von der Benutzeroberfläche bleibt.

5. Bausteinsicht

5.1 Paketdiagramm



Beschreibung Paketdiagramm

Das Paketdiagramm zeigt die Architektur von SortiFX in vier übereinander angeordneten Schichten: **view**, **controller**, **model** und **persistence**. Die Abhängigkeiten verlaufen ausschließlich von oben nach unten, kein unteres Paket kennt ein oberes.

Paket **view** enthält alle Klassen der grafischen Benutzeroberfläche. Die drei Hauptansichten *MainView*, *EditorView* und *SimulationsView* bilden die primären Bildschirmansichten der Anwendung. Das Unterpaket **dialog** fasst die modalen Dialoge *GepaeckDialog*, *WeichenDialog* und *SpeicherDialog* zusammen, die kontextabhängig aus den Views heraus geöffnet werden. Der <<access>>-Pfeil von **view** zum Unterpaket **dialog** zeigt, dass die Views auf diese Dialoge zugreifen können.

Paket **controller** enthält die Steuerungsklassen, die zwischen Oberfläche und Modell vermitteln. Jede View besitzt einen zugehörigen Controller: *MainView* → *MainMenuController*, *EditorView* → *EditorController*, *SimulationsView* → *SimulationsController*. Der *SpeicherController* wird vom *SpeicherDialog* aus angesteuert und koordiniert das Speichern und Laden über die Persistenzschicht. Die gestrichelten Abhängigkeitspfeile von **view** nach **controller** zeigen, dass die Views ihre Controller kennen und aufrufen, aber nicht umgekehrt.

Paket **model** enthält sowohl die Domänenklassen als auch die UI-neutrale Logik der Anwendung. *AnlagenPlan* ist die zentrale Datenklasse, die das gesamte Modell einer Förderanlage repräsentiert. *Raster* und *Foerderelement* bilden die Grundstruktur der Anlage, *Gepaeckstueck* repräsentiert die simulierten Objekte. Die drei Logikklassen *Validierungslogik*, *Simulationslogik* und *Routinglogik* kapseln die Kernfunktionen der Anwendung vollständig. *EditorController* greift auf *Foerderelement* und *Validierungslogik* zu, *SimulationsController* auf *Simulationslogik* und *Routinglogik*, *MainMenuController* auf *AnlagenPlan*.

Paket **persistence** übernimmt die Serialisierung und Deserialisierung des Anlagenplans. *AnlagenplanRepository* stellt die Schnittstelle zum Dateisystem bereit und nutzt *Speicherformat* zur konkreten Formatierung der Daten. Der *SpeicherController* aus der **Controller**-Schicht greift auf *AnlagenplanRepository* zu, das seinerseits mit den Domänenklassen im **model** arbeitet.

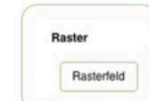
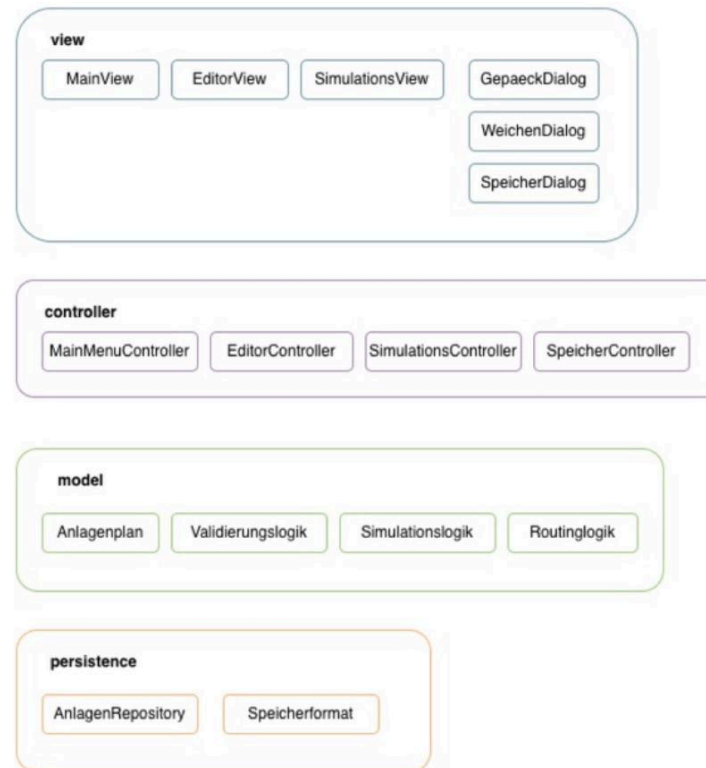
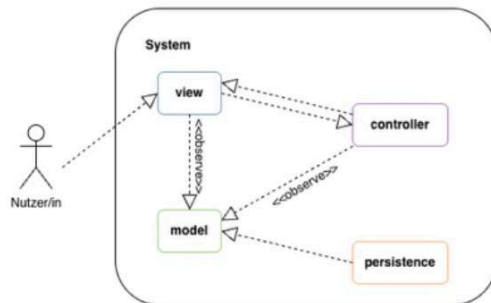
5. 2 Verfeinerungsebenen

Ebene 1

Ebene 2

Ebene 3

Ebene 4



Beschreibung Verfeinerungsebenen

Die Bausteinsicht beschreibt die statische Struktur von SortiFX in fünf Verfeinerungsebenen. Ausgangspunkt ist das Gesamtsystem als Black Box, das schrittweise bis auf die Ebene konkreter Domänenklassen verfeinert wird.

Ebene 1: Systemüberblick

Das System SortiFX wird von einer Nutzerin oder einem Nutzer bedient und besteht intern aus vier Paketen: view, controller, model und persistence. Die Abhängigkeiten folgen dem MVC-Architekturmuster: Die view leitet Nutzereingaben an den controller weiter, der controller manipuliert das model und koordiniert die persistence. Das model benachrichtigt die view über Änderungen mittels Observer-Pattern («observe»). Kein unteres Paket kennt ein oberes — das model hat keine Abhängigkeit zu controller oder view.

Ebene 2: Verfeinerung der Pakete

Das Paket view enthält die drei Hauptansichten MainView, EditorView und SimulationsView sowie die modalen Dialoge GepaeckDialog, WeichenDialog und SpeicherDialog, die kontextabhängig geöffnet werden.

Das Paket controller enthält je einen Controller pro Hauptansicht: MainMenuController, EditorController und SimulationsController. Der SpeicherController koordiniert das Speichern und Laden von Anlagenplänen.

Das Paket model enthält die zentrale Klasse AnlagenPlan sowie die drei UI-neutralen Logikklassen Validierungslogik, Simulationslogik und Routinglogik. Diese Klassen kapseln die Kernfunktionen der Anwendung vollständig ohne JavaFX-Abhängigkeiten und sind damit unabhängig mit JUnit testbar.

Das Paket persistence enthält AnlagenRepository zur Verwaltung von Speicher- und Ladevorgängen sowie Speicherformat, das das konkrete Dateiformat kapselt.

Ebene 3: Verfeinerung model: AnlagenPlan

AnlagenPlan ist die zentrale Datenklasse und fasst die gesamte Modellierung einer Förderanlage zusammen. Er enthält ein Raster, das die zweidimensionale Arbeitsfläche der Anwendung repräsentiert. Diese Kompositionsbeziehung bedeutet, dass das Raster nicht unabhängig vom AnlagenPlan existieren kann.

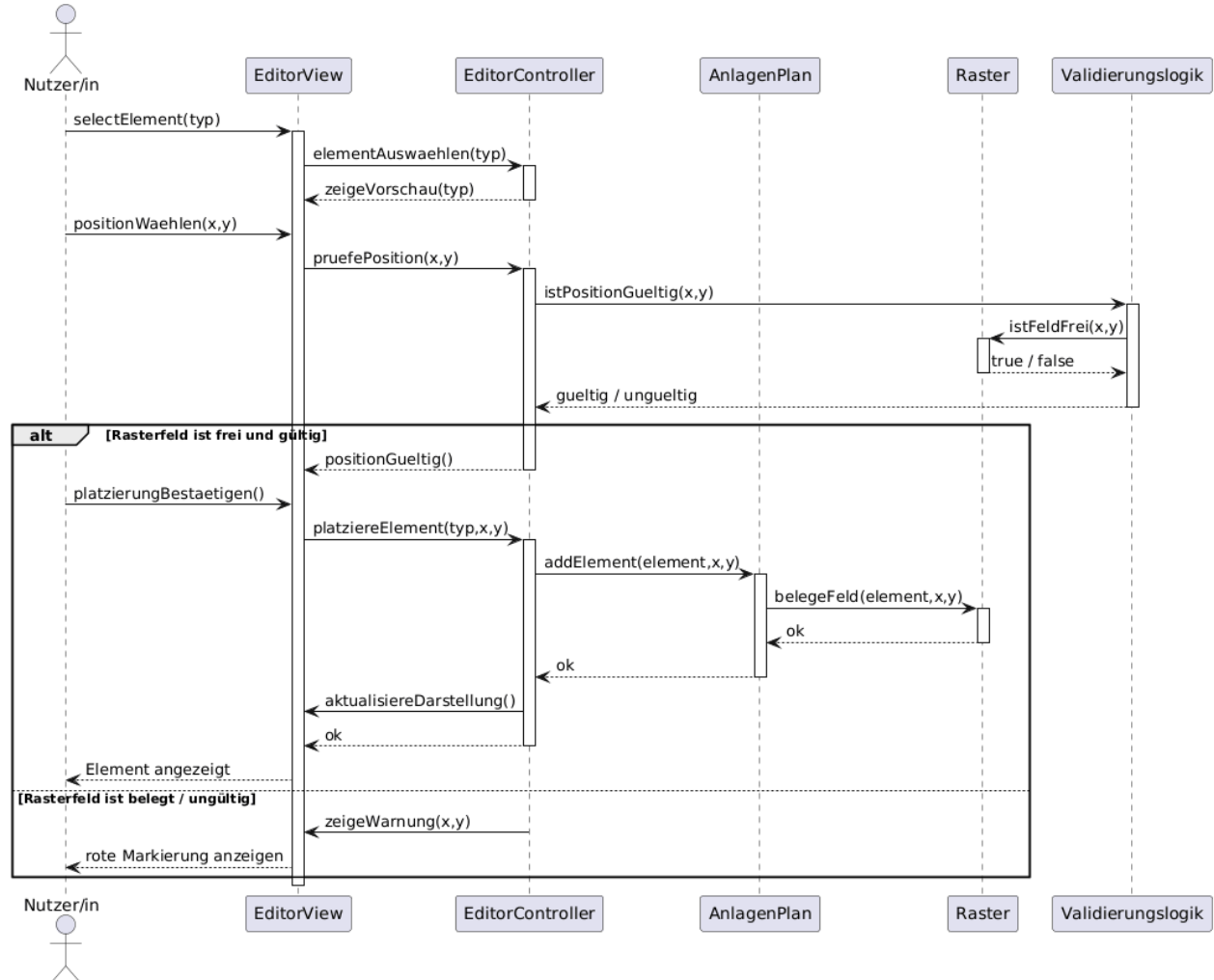
Ebene 4: Verfeinerung Raster

Das Raster besteht aus einer Menge von Rasterfeld-Objekten, die jeweils eine Position auf der Arbeitsfläche repräsentieren und angeben, ob das Feld belegt ist. Auch hier handelt es sich um eine Komposition: Rasterfeld-Objekte sind fester Bestandteil des Rasters und können nicht isoliert existieren.

Die weitere Verfeinerung der Domänenklassen — insbesondere die Vererbungshierarchie von Foerderelement mit den Unterklassen Gerade, Knick, Weiche und Zielstation sowie die Komposition Weiche → WeichenRegel — ist im Domänenmodell der Anforderungsspezifikation (Kapitel 3.1) vollständig dokumentiert und wird hier nicht erneut abgebildet.

6. Laufzeitsicht

6.1 Szenario: Förderband-Element platzieren



Das Sequenzdiagramm zeigt den Ablauf der Platzierung eines Förderband-Elements. Zunächst wählt die **Nutzerin** bzw. der **Nutzer** in der **EditorView** ein Element aus. Der **EditorController** veranlasst daraufhin die Anzeige einer Vorschau.

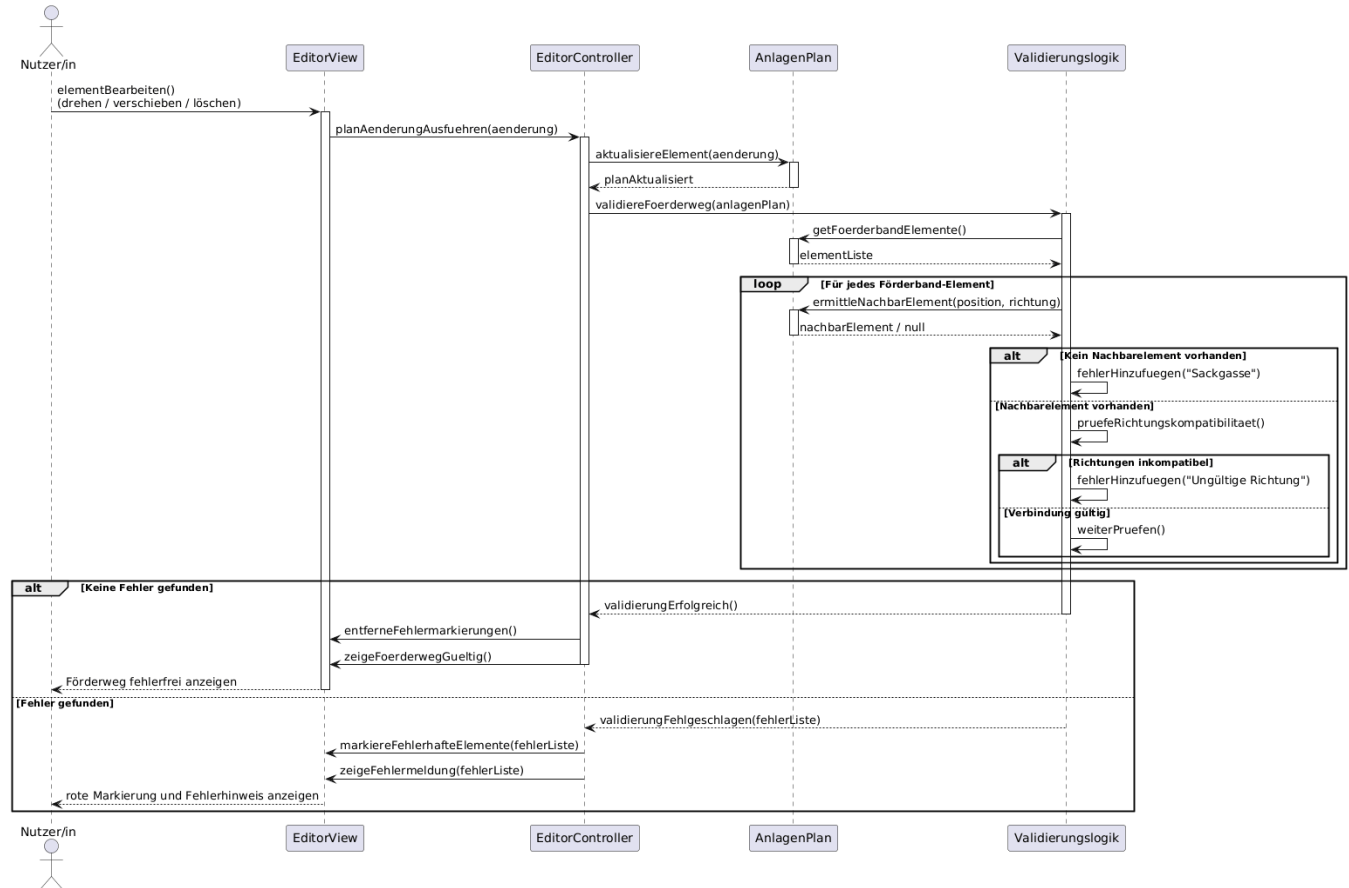
Anschließend wird eine Position im **Raster** ausgewählt. Der **EditorController** übergibt die Prüfung an die **Validierungslogik**, welche mithilfe des **Rasters** überprüft, ob das ausgewählte Feld frei und gültig ist.

Ist die Position gültig, bestätigt die Nutzerin bzw. der Nutzer die Platzierung. Das Element wird dem **Anlagenplan** hinzugefügt, das **Rasterfeld** belegt und die Darstellung der Benutzeroberfläche aktualisiert.

Ist das Feld bereits belegt oder ungültig, wird eine visuelle Warnung angezeigt. Die Nutzerin bzw. der Nutzer kann anschließend eine neue Position auswählen, worauf die Prüfung erneut durchgeführt wird.

Das Diagramm verdeutlicht das Zusammenspiel von View, Controller, Fachlogik und Datenmodell bei der Platzierung eines Förderband-Elements.

6.2 Szenario: Förderweg validieren



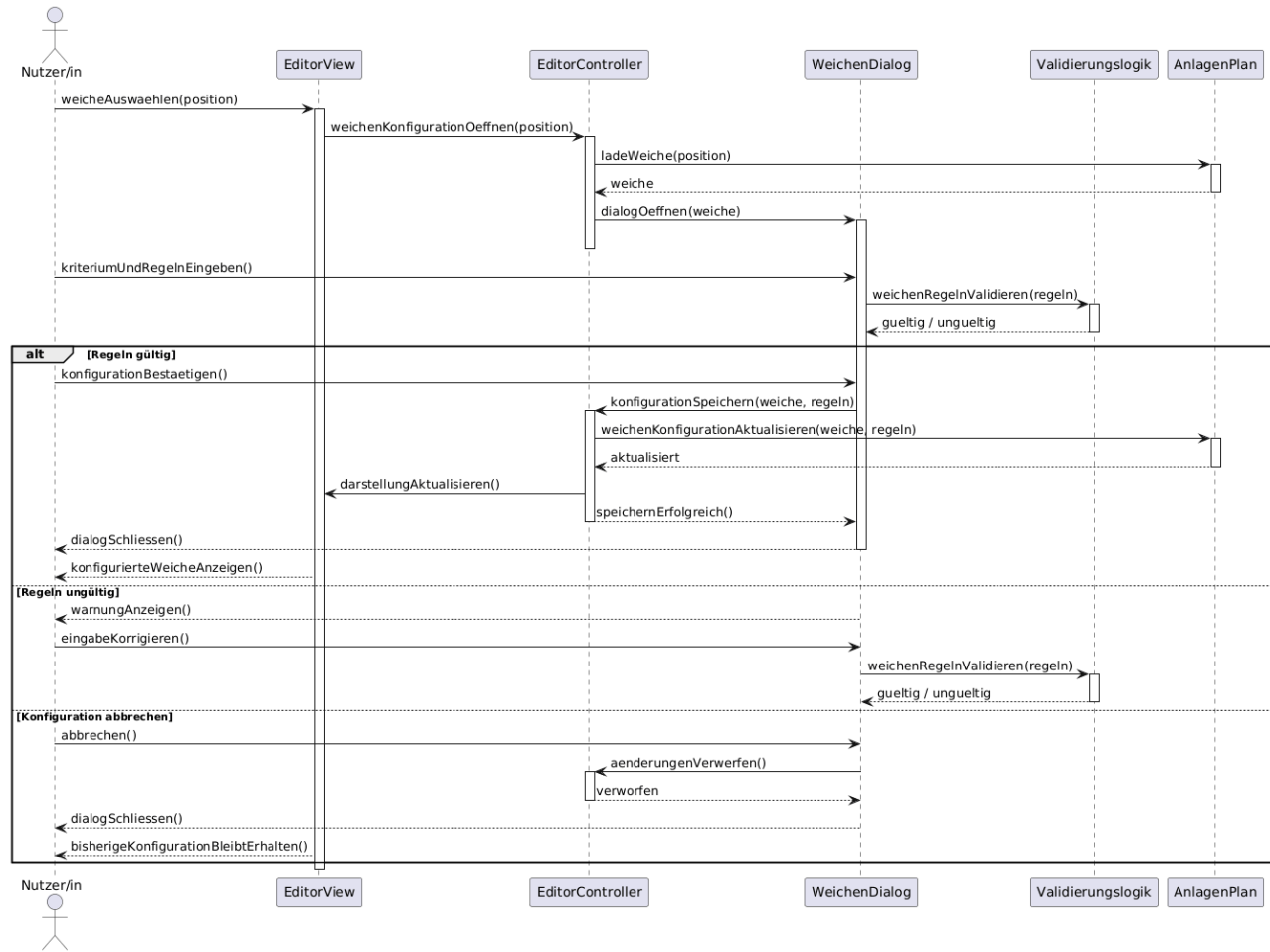
Das Sequenzdiagramm zeigt die automatische Validierung des Förderwegs nach einer Änderung am Anlagenplan. Sobald die Nutzerin bzw. der Nutzer ein Förderband-Element dreht, verschiebt oder löscht, wird diese Änderung über die **EditorView** an den **EditorController** weitergegeben.

Der **EditorController** aktualisiert zunächst den **AnlagenPlan** und startet anschließend die **Validierungslogik**. Diese prüft für jedes Förderband-Element, ob ein gültiges Nachbarelement vorhanden ist und ob die Förderrichtungen der verbundenen Elemente zueinander passen.

Wird kein gültiges Nachbarelement gefunden, entsteht eine Sackgasse. Stimmen die Richtungen zweier Elemente nicht überein, wird eine ungültige Verbindung erkannt. In beiden Fällen sammelt die **Validierungslogik** entsprechende Fehler.

Nach Abschluss der Prüfung gibt die **Validierungslogik** das Ergebnis an den **EditorController** zurück. Sind keine Fehler vorhanden, entfernt die **EditorView** vorhandene Fehlermarkierungen und zeigt den Förderweg als gültig an. Werden Fehler gefunden, markiert die Oberfläche die betroffenen Elemente und zeigt einen entsprechenden Fehlerhinweis.

6.3 Szenario: Weiche konfigurieren



Das Sequenzdiagramm zeigt den Ablauf zur Konfiguration einer bereits platzierten Weiche. Die Nutzerin bzw. der Nutzer wählt eine Weiche in der **EditorView** aus. Der **EditorController** lädt die zugehörige **Weiche** aus dem **AnlagenPlan** und öffnet den **WeichenDialog**.

Im Dialog werden Entscheidungskriterium und Weiterleitungsregeln eingegeben. Die **Validierungslogik** prüft, ob die Regeln vollständig und widerspruchsfrei sind. Sind die Regeln gültig, speichert der **EditorController** die Konfiguration im **AnlagenPlan** und aktualisiert die Darstellung.

Bei ungültigen Regeln zeigt der Dialog eine Warnung an und die Eingabe kann korrigiert werden. Wird der Vorgang abgebrochen, werden die Änderungen verworfen und die bisherige Weichenkonfiguration bleibt erhalten.

7. Verteilungssicht

SortiFX wird als lokale Desktop-Anwendung ausgeführt.

Bestandteile:

- JavFX Benutzeroberfläche
- Controller-Schicht
- Fachlogik
- Datenspeicherung

Alle Komponenten laufen innerhalb eines Prozesses auf demselben Rechner. Externe Server oder Netzwerkkommunikation werden nicht verwendet.

8. Querschnittliche Konzepte

8.1 MVC

- Model, View, Controller
- siehe 5. Bausteinsicht

8.2 Validierungskonzept

- siehe Use Case: Flrderweg validieren (Anforderungsspezifikation Kapitel 2.2.2 Fall 4)
- automatische Prüfung, Sackgassen, Richtungsfehler, Schleifen

8.3 Persistenzkonzept

- Speicherung als Datei
- Laden aus Datei

9. Entwurfsentscheidungen

9.1 MVC

SortiFX wurde nach dem MVC-Prinzip (Model-View-Controller) aufgebaut. Die Benutzeroberfläche (View), die Steuerungslogik (Controller) und das fachliche Datenmodell (Model) sind klar voneinander getrennt. Dadurch bleibt die Anwendung übersichtlich, leichter wartbar und die Kernlogik kann unabhängig von der grafischen Oberfläche getestet werden.

9.2 JavaFX

Die grafische Benutzeroberfläche von SortiFX wird mit JavaFX umgesetzt. JavaFX wurde gewählt, da es die im Praktikum geforderte Technologie ist und sich gut für die Entwicklung moderner Desktop-Anwendungen eignet. Darüber hinaus ermöglicht JavaFX die einfache Umsetzung der rasterbasierten Modellierungsoberfläche sowie der animierten Simulation von Gepäckstücken.

9.3 Validierung als eigene Klasse

Die Überprüfung von Förderwegen wurde in einer eigenen Klasse „Validierungslogik“ umgesetzt. Dadurch bleibt die Validierung unabhängig von der Benutzeroberfläche und kann zentral von verschiedenen Anwendungsfällen genutzt werden. Gleichzeitig verbessert diese Trennung die Testbarkeit und erleichtert spätere Erweiterungen um zusätzliche Validierungsregeln, beispielsweise für neue Förderband-Elemente oder komplexere Förderwege.

10. Qualitätsszenarien

Siehe Kapitel 4.1 der Anforderungsspezifikation.

11. Risiken & technische Schulden

Risiken

- Simulationslogik könnte bei großen Anlagen langsam werden.
- Schleifenerkennung noch nicht vollständig getestet.
- Speicherung noch nicht auf große Dateien optimiert.

technische Schulden

- Die Validierungslogik prüft aktuell nur die wichtigsten Fehlerfälle (z.B. Sackgassen und ungültige Richtungen). Komplexere Prüfungen könnten künftig ergänzt werden.
- Die Speicherung erfolgt derzeit nur lokal über Dateien. Eine Datenbankbindung wurde nicht berücksichtigt.
- Die Benutzeroberfläche wurde primär für Desktop-Nutzung entwickelt und nicht für verschiedene Bildschirmgrößen optimiert.

12. Glossar

Begriff	Beschreibung
Anlagenplan	Zentrale Datenklasse der Anwendung, die die gesamte modellierte Gepäckförderanlage enthält.
Architekturdokument	Dokument, das den technischen Aufbau, die Struktur und das Zusammenspiel der wichtigsten Bestandteile der Anwendung beschreibt.
Bausteinsicht	Sicht auf die statische Struktur der Software, z. B. Pakete, Klassen und deren Abhängigkeiten.
Controller	Vermittelt zwischen Benutzeroberfläche und fachlicher Logik. Er nimmt Benutzereingaben entgegen und ruft passende Funktionen im Modell auf.
Controller-Schicht	Schicht der Anwendung, die Eingaben aus der View verarbeitet und Abläufe zwischen Oberfläche, Modell und Persistenz koordiniert.
Dialog	Modales Fenster zur Eingabe oder Bearbeitung bestimmter Daten, z. B. Gepäckdaten, Weichenregeln oder Speicherdaten.
Domänenmodell	Modell der wichtigsten fachlichen Objekte der Anwendung, z. B. Anlagenplan, Raster, Gepäckstück oder Förderelement.
EditorController	Controller, der Aktionen im Editor verarbeitet, z. B. Platzieren, Drehen, Verschieben oder Löschen von Förderelementen.
EditorView	Ansicht, in der Nutzerinnen und Nutzer den Anlagenplan bearbeiten und Förderelemente platzieren können.
Förderband-Element	Baustein einer Gepäckförderanlage, z. B. Gerade, Knick, Weiche oder Zielstation.
Förderrichtung	Richtung, in die ein Förderband-Element ein Gepäckstück weitertransportiert.
Förderweg	Zusammenhängende Folge von Förderband-Elementen, entlang derer Gepäckstücke transportiert werden.
GepaeckDialog	Dialog zur Erstellung oder Bearbeitung eines Gepäckstücks.

Gepäckstück	Simuliertes Objekt, das sich während der Simulation durch die Förderanlage bewegt.
JavaFX	Framework zur Entwicklung grafischer Desktop-Anwendungen in Java.
Kontextabgrenzung	Beschreibung, welche externen Akteure oder Systeme mit SortiFX interagieren.
Laufzeitsicht	Sicht auf dynamische Abläufe im System, z. B. als Sequenzdiagramm für einen bestimmten Use Case.
Model	Paket bzw. Schicht mit den fachlichen Datenstrukturen und der UI-neutralen Kernlogik der Anwendung.
Model-Logik	Fachliche Logik innerhalb des Models, z. B. Validierungslogik, Simulationslogik oder Routinglogik.
MVC	Architekturmuster, das die Anwendung in View, Controller und Model trennt.
Nutzer/in	Person, die SortiFX bedient und Anlagenpläne erstellt, bearbeitet oder simuliert.
Paketdiagramm	Diagramm zur Darstellung der Pakete einer Software und ihrer Abhängigkeiten.
Persistence	Schicht, die für das Speichern und Laden von Anlagenplänen zuständig ist.
Repository	Klasse, die den Zugriff auf gespeicherte Daten kapselt, z. B. das Laden und Speichern eines Anlagenplans.
Routinglogik	Logik, die bei Weichen entscheidet, in welche Richtung ein Gepäckstück weitergeleitet wird.
Routingregel	Regel, nach der eine Weiche ein Gepäckstück abhängig von Eigenschaften wie Zielort oder Gewicht weiterleitet.
Simulation	Virtuelle Ausführung des Gepäcktransports innerhalb des Anlagenplans.
Simulationslogik	UI-neutrale Logik, die die Bewegung der Gepäckstücke während der Simulation berechnet.

SimulationController	Controller, der Simulationsaktionen wie Starten, Pausieren oder Stoppen verarbeitet.
SimulationView	Ansicht, in der der Simulationszustand und die Bewegung der Gepäckstücke dargestellt werden.
SpeicherDialog	Dialog zur Auswahl eines Dateinamens oder Speicherorts beim Speichern eines Anlagenplans.
Speicherformat	Struktur, in der ein Anlagenplan gespeichert und später wieder geladen werden kann.
Validierungslogik	Logik zur Prüfung des Anlagenplans auf Fehler wie Sackgassen, ungültige Verbindungen oder inkompatible Richtungen.
View	Schicht der Anwendung, die die grafische Benutzeroberfläche darstellt und Eingaben an Controller weiterleitet.
WeichenDialog	Dialog zur Konfiguration einer Weiche und ihrer Routingregeln.